

{ key : value }

Diccionarios en Python

Creación, iteración y métodos más comunes

Skill: Introducción a la Programación

Docente: Jhonathan Arley Peñaloza Sierra

Campuslands · 2026

Objetivos de Aprendizaje

- 1 Comprender qué es un diccionario y cuándo usarlo
- 2 Crear diccionarios de diferentes formas
- 3 Acceder y modificar elementos por su llave (key)
- 4 Iterar un diccionario recorriendo keys, values o ambos
- 5 Dominar los métodos más comunes de diccionarios

¿Qué es un Diccionario?



Piensa en un diccionario real...

Un diccionario de español tiene:

- Una palabra (la buscas)
- Su significado (lo que encuentras)

Buscas por la PALABRA
y obtienes el SIGNIFICADO.



En Python es igual...

Un diccionario de Python tiene:

- Una llave (key) → la buscas
- Un valor (value) → lo que obtienes

Buscas por la KEY
y obtienes el VALUE.



Definición:

Un diccionario es una estructura de datos que almacena pares de llave:valor. Cada llave es única y sirve para acceder rápidamente a su valor asociado. Se crean con llaves { }.

```
d = {'color': 'blanco'} → d['color'] # → 'blanco'
```

Crear un Diccionario

Forma 1: Con llaves { }

La más común y recomendada.

```
usuario = {  
    "Nombre": "Sara",  
    "Edad": 27,  
    "Documento": 1003882  
}
```

Forma 2: Con dict()

Usando el constructor.

```
usuario = dict(  
    Nombre='Sara',  
    Edad=27,  
    Documento=1003882  
)
```

Propiedades de los diccionarios:



Dinámicos

Crecen o decrecen, se añaden o eliminan elementos



Indexados por key

Se accede a los valores a través de la llave

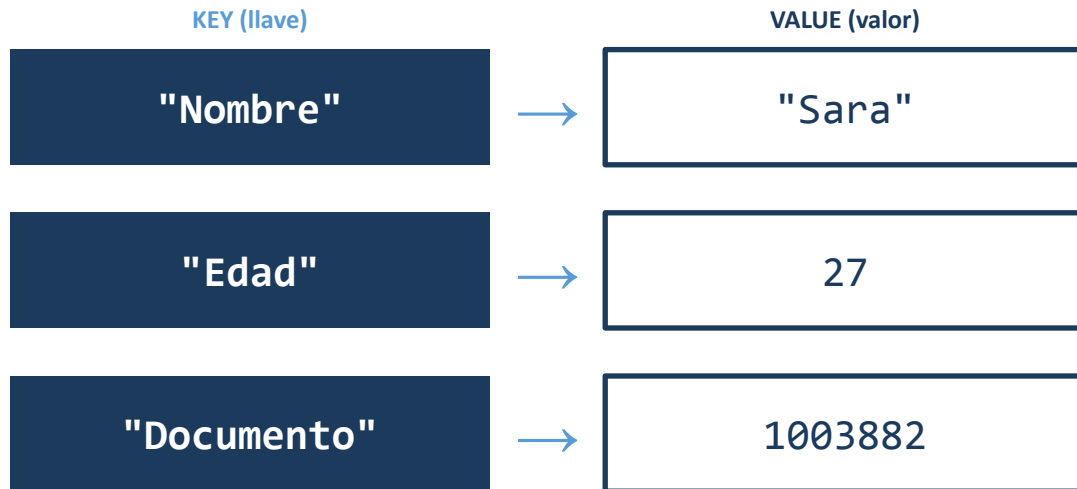


Anidados

Un diccionario puede contener otros diccionarios

Estructura: Llave → Valor

Cada par key:value es como una etiqueta y su contenido:



Recuerda:

- Las keys son únicas
- Los valores pueden repetirse
- Las keys son strings o números
- Los valores pueden ser cualquier tipo

 Pregunta: ¿Qué pasa si intentas crear un diccionario con dos keys iguales? Ejemplo: {"a": 1, "a": 2}

Acceder y Modificar Elementos

Acceder a un valor

Dos formas equivalentes:

```
# Con corchetes
print(usuario['Nombre']) # Sara
# Con get()
print(usuario.get('Nombre')) # Sara
```

Modificar un valor

Asignar un nuevo valor a una key:

```
# Cambiar valor existente
usuario["Nombre"] = "Laura"
# Agregar nueva key (si no existe)
usuario["Direccion"] = "Calle 123"
```

Si asignas un valor a una key que NO existe, se añade automáticamente al diccionario.



Actividad rápida:

Crea un diccionario "mascota" con keys: nombre, tipo, edad. Luego cambia la edad y agrega una nueva key "color".

Iterar un Diccionario

Solo las KEYS

```
for clave in usuario:  
    print(clave)  
# Nombre  
# Edad  
# Documento
```

Solo los VALUES

```
for clave in usuario:  
    print(usuario[clave])  
# Sara  
# 27  
# 1003882
```

KEYS y VALUES juntos

```
for k, v in usuario.items():  
    print(k, v)  
# Nombre Sara  
# Edad 27  
# Documento 1003882
```



El método `.items()` es el más útil: te da AMBOS (key y value) en cada iteración.

Métodos: Consultar y Limpiar

get(key, default)

Obtiene el valor de una key. Si no existe, devuelve el default.

```
d = {"a": 1, "b": 2}
d.get("a")      # → 1
d.get("z", "No") # → "No"
```

keys()

Devuelve todas las llaves del diccionario.

```
d = {"a": 1, "b": 2}
print(d.keys())
# dict_keys(['a', 'b'])
```

values()

Devuelve todos los valores del diccionario.

```
d = {"a": 1, "b": 2}
print(d.values())
# dict_values([1, 2])
```

clear()

Elimina TODOS los elementos del diccionario.

```
d = {"a": 1, "b": 2}
d.clear()
print(d) # {}
```


Métodos: Modificar y Eliminar

items()

Devuelve pares (key, value) como tuplas.

```
d = {"a": 1, "b": 2}
print(list(d.items()))
# [('a',1), ('b',2)]
```

pop(key, default)

Elimina la key y devuelve su valor.

```
d = {"a": 1, "b": 2}
d.pop("a")
print(d) # {'b': 2}
```

popitem()

Elimina el último par insertado.

```
d = {"a": 1, "b": 2}
d.popitem()
print(d) # {'a': 1}
```

update(otro_dict)

Actualiza con otro diccionario. Agrega keys nuevas.

```
d1 = {"a":1, "b":2}
d2 = {"a":0, "d":400}
d1.update(d2)
# {'a':0, 'b':2, 'd':400}
```

Resumen de Métodos

Método	¿Qué hace?	Ejemplo rápido
<code>get(key)</code>	Obtiene valor por key	<code>d.get("a") → 1</code>
<code>keys()</code>	Devuelve todas las keys	<code>d.keys() → ['a', 'b']</code>
<code>values()</code>	Devuelve todos los values	<code>d.values() → [1, 2]</code>
<code>items()</code>	Devuelve pares (key, value)	<code>d.items() → [('a', 1)]</code>
<code>pop(key)</code>	Elimina y devuelve valor	<code>d.pop('a') → 1</code>
<code>popitem()</code>	Elimina último par	<code>d.popitem()</code>
<code>update(d2)</code>	Actualiza con otro dict	<code>d.update({'c': 3})</code>
<code>clear()</code>	Elimina todo	<code>d.clear() → {}</code>

Diccionarios Anidados

Un diccionario puede contener OTROS diccionarios como valores. Esto permite estructuras más complejas.

```
productos = {  
    "Menta": {  
        "costo": 40,  
        "precio": 300,  
        "cantidadDisponible": 10  
    },  
    "Chocorrano": {  
        "costo": 700,  
        "precio": 1000,  
        "cantidadDisponible": 12  
    }  
}
```



¿Cómo acceder?

```
# Precio de la Menta  
productos["Menta"]["precio"]  
# → 300
```

```
# Cantidad de Chocorrano  
productos["Chocorrano"]  
    ["cantidadDisponible"]  
# → 12
```



Se accede con dos []: el primero selecciona el producto, el segundo la propiedad.

Ejemplo 1: La Tienda de Johan

 Problema: Preguntar al usuario un producto y cantidad, mostrar el total.

```
productos = {  
    "Menta": 300, "Chocorrano": 1000,  
    "Esfero": 2700, "Chocolatina": 2500  
}  
  
producto = input('¿Qué producto? ').title()  
cantidad = int(input('¿Cuántos? '))  
  
if producto in productos:  
    total = productos[producto] * cantidad  
    print(f"Total: ${total}")  
else:  
    print("Producto no disponible")
```

¿Qué hace?

1. Crea el diccionario de productos
2. Pide producto y cantidad
3. Verifica si existe con in
4. Calcula precio × cantidad
5. Si no existe, avisa al usuario

Ejemplo 2: Inventario de la Tienda

 Problema: Mostrar cantidades de cada producto y el total general.

```
productos = {  
    "Mentas": 10, "Chocorramos": 12,  
    "Esferos": 2, "Chocolatinas": 4  
}  
  
total_productos = 0  
for producto, cantidad in productos.items():  
    print(f"{producto}: {cantidad} unidades")  
    total_productos += cantidad  
  
print(f"Total: {total_productos} productos")
```

Resultado:

Mentas: 10 unidades
Chocorramos: 12 unidades
Esferos: 2 unidades
Chocolatinas: 4 unidades

Total: 28 productos

 Usamos `.items()` para obtener producto y cantidad en cada iteración, y un acumulador para el total.



Retos para Practicar

Usa este diccionario anidado para los 3 retos:

```
productos = {  
  'Menta': {'costo':40, 'precio':300, 'cantidadDisponible':10},  
  'Chocorramo': {'costo':700, 'precio':1000, 'cantidadDisponible':12}  
}
```

Reto 1:

Adapta los ejemplos de la tienda de Johan y el inventario para que funcionen con este diccionario anidado (accede a precio con productos[producto]['precio']).

Reto 2:

Cuando el cliente pide un producto, descuenta de cantidadDisponible si hay suficiente stock. Si no hay suficiente, muestra un mensaje. Al final imprime el diccionario actualizado.

Reto 3:

Calcula la ganancia por producto si se venden TODOS:
 $\text{ganancia} = (\text{precio} - \text{costo}) \times \text{cantidadDisponible}$
Muestra la ganancia de cada producto y la ganancia total.



Tiempo: 20 minutos | Trabaja en parejas

Diccionario vs Lista: ¿Cuándo usar cada uno?

Característica	Lista []	Diccionario { }
Acceso	Por índice numérico (0, 1, 2...)	Por llave (key) con nombre
Orden	Mantiene el orden	Mantiene orden de inserción
Búsqueda	Lenta (recorre todo)	Rápida (busca por key)
Uso ideal	Secuencias de datos similares	Datos con etiquetas/nombre
Ejemplo	notas = [80, 90, 70]	persona = {"edad": 25}



Regla sencilla:

¿Tus datos tienen etiquetas? → Diccionario. ¿Son una secuencia sin nombre? → Lista.



Actividad Práctica

Resuelve estos ejercicios en Python:

1. Crea un diccionario con tus datos: nombre, edad, ciudad, hobby
2. Accede al nombre con `[ ]` y al hobby con `.get()`
3. Agrega una nueva key "correo" con tu email
4. Recorre el diccionario e imprime cada key y value con `.items()`
5. Usa `.keys()` y `.values()` para imprimir solo las llaves y solo los valores
6. Elimina la key "hobby" con `.pop()` e imprime el resultado



Tiempo: 10 minutos



Resumen de la Clase



Un diccionario almacena pares key:value — como un diccionario real



Se crean con { } o dict() — cada key es única



Se accede con [] o .get() — se modifica asignando a la key



Se itera con for: keys, values o .items() para ambos



Métodos clave: get, keys, values, items, pop, update, clear



Pueden ser anidados: un diccionario dentro de otro

¡Los diccionarios son la estructura perfecta para datos con etiquetas! 🚀

¿Preguntas?

"La llave correcta abre cualquier puerta"
— *La clave de los diccionarios*